

TABLE III
PERFORMANCE ACROSS DIFFERENT MODELS AND CONFIGURATIONS ON NYU CTF BENCH, CYBENCH, AND HACKTHEBOX BENCHMARKS.

	NYU CTF Bench								Cybench		HackTheBox	
	% solved	\$ cost	crypto	forensics	pwn	rev	web	misc	% solved	\$ cost	% solved	\$ cost
NYU CTF baseline												
Claude 3.5 Sonnet	13.0	–	7.7	20.0	7.7	21.6	5.3	16.7	15.0	–	38.0	–
GPT 4o	6.0	–	3.8	0.0	5.1	9.8	0.0	12.5	12.5	–	16.0	–
GPT 4 Turbo	6.0	–	1.9	0.0	5.1	9.8	0.0	16.7	12.5	–	10.0	–
EnIGMA												
Claude 3.5 Sonnet	13.5	0.35	7.7	20.0	18.0	17.7	0.0	16.7	20.0	0.91	26.0	0.53
GPT 4o	9.5	0.62	3.9	13.3	7.7	13.7	5.3	16.7	12.5	0.61	16.3	1.71
GPT 4 Turbo	7.0	0.79	1.9	13.3	5.1	9.8	0.0	16.7	17.5	1.60	18.4	1.35
D-CIPHER												
Claude 3.5 Sonnet	19.0	0.52	15.4	20.0	12.8	29.4	5.3	25.0	22.5	0.30	44.0	0.49
GPT 4o	10.5	0.22	5.8	13.3	7.7	13.7	10.5	16.7	12.5	0.08	16.0	0.16
GPT 4 Turbo	6.5	0.46	1.9	13.3	5.1	7.8	5.3	12.5	–	–	–	–
LLaMa 3.1 405B	3.0	0.01	1.9	0.0	0.0	3.9	0.0	12.5	–	–	–	–
Gemini 1.5 Flash	2.5	0.001	1.9	0.0	0.0	3.9	0.0	8.3	–	–	–	–
D-CIPHER w/o auto-prompter												
Claude 3.5 Sonnet	22.0	0.74	15.4	20.0	28.2	27.5	10.5	25.0	20.0	0.33	44.0	0.62
GPT 4o	9.5	0.23	1.9	6.7	5.1	17.6	10.5	16.7	–	–	–	–
D-CIPHER w/o planner												
Claude 3.5 Sonnet	14.0	0.36	9.6	6.7	7.7	25.5	5.3	20.8	–	–	–	–
GPT 4o	9.0	0.11	3.8	6.7	5.1	13.7	5.3	20.8	–	–	–	–

reflects the category-wise accuracy and offers granular insights into offensive capability.

V. RESULTS

A. Comparison of % solved

Table III compares the performance of D-CIPHER with other LLM agents across multiple LLMs and benchmarks. We run D-CIPHER with five different LLMs, using the same LLM for Planner, Executor, and Auto-prompter in each run. We also rerun the NYU CTF baseline agent with three LLMs to measure the impact of recent updates to the LLM models on NYU CTF Bench. The EnIGMA % *solved* and \$ *cost* are taken from Abramovich et al. [1]. As EnIGMA sets a new benchmark on Cybench with state-of-the-art results, we do not include comparisons with the earlier baseline agent [49].

D-CIPHER with Claude 3.5 Sonnet consistently outperforms the current state-of-the-art EnIGMA, achieving 19.0% over 13.5% on NYU CTF Bench, 22.5% over 20% on Cybench, and 44% over 26% on HackTheBox. D-CIPHER with GPT 4o also outperforms EnIGMA with GPT 4o on NYU CTF Bench, while getting a close result on Cybench and HackTheBox. The rerun results of NYU CTF baseline show that recent LLM models have improved on cybersecurity tasks, getting close to EnIGMA’s state-of-the-art performance. Yet, D-CIPHER consistently beats the baseline on NYU CTF Bench in overall % *solved* for both Claude 3.5 Sonnet and GPT 4o. EnIGMA was evaluated with older versions of LLMs, so we evaluate D-CIPHER with the older Claude 3.5 Sonnet to show that we still outperform (see Section V-D4).

These results indicate that D-CIPHER improves capabilities across multiple LLM architectures, and the higher performance stems not only from recent LLM updates but also from its multi-agent system architecture. Interestingly, D-CIPHER without Auto-prompter with Claude 3.5 Sonnet achieves the highest performance of 22% on NYU CTF Bench. However,

performance without the Auto-prompter worsens on GPT 4o and on other benchmarks, while average cost increases, indicating that the Auto-prompter helps overall.

B. Comparison of \$ cost

Table III also compares average \$ *cost* of solved challenges. Except for Claude 3.5 Sonnet on NYU CTF Bench, D-CIPHER has a lower average cost across all LLMs and benchmarks. With GPT 4o and GPT 4 Turbo, D-CIPHER lowers the cost by 2× to 10× across benchmarks while solving more challenges. Despite having multiple agents, a significant cost reduction indicates that division of responsibilities between agents makes the problem-solving system more efficient.

C. Category-wise comparison

Table III shows the categorywise % *solved* of D-CIPHER and other works on NYU CTF Bench. EnIGMA’s results are computed from their provided transcripts, while NYU CTF baseline’s results are computed from our reruns. D-CIPHER’s performance improvement stays consistent across the CTF categories. D-CIPHER outperforms EnIGMA across all categories except pwn, with a notable improvement in crypto, where its performance doubles from 7.7% to 15.4%. Likewise, on rev, and misc, we see a 9%–12% increase. The improvement is due to the enhanced task decomposition and execution ability of the Planner-Executor system. Especially, crypto and rev frequently have long outputs of disassembled binaries or encrypted files requiring multiple analysis steps that are decomposed by the Planner.

Figure 5(a) plots the % *solved* of D-CIPHER across categories on NYU CTF Bench. D-CIPHER’s performance is more balanced across different LLMs, demonstrating that our framework operates well with different reasoning capabilities of the LLMs. While D-CIPHER improves in web over previous results, the performance still lags behind other categories,

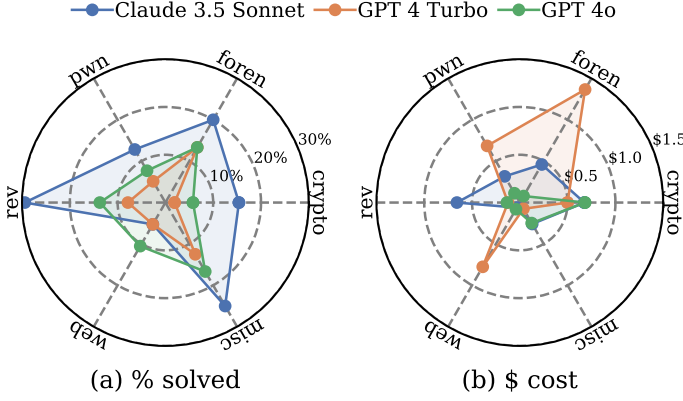


Fig. 5. % solved by category for D-CIPHER on NYU CTF Bench.

pointing to a common limitation in web CTFs. Figure 5(b) plots the average \$ cost. GPT 4o is most cost efficient across categories, while Claude 3.5 Sonnet is moderately higher on forensics, pwn, and rev. GPT 4 Turbo is the costliest among the three LLMs, for forensics, pwn and web, while on other categories has a lower cost but also solves less challenges. crypto has higher cost across LLMs as it may require analysis of long encrypted texts, and many iterations for decryption.

D. Impact of different configurations

1) *Ablation Study*: D-CIPHER’s special interaction functions allow versatility to configure different types of multi-agent systems. We run D-CIPHER with two different configurations: (1) without the Auto-prompter where the hard-coded prompt template is used for the Planner’s initial prompt, and (2) without the Planner where a single Executor is run with the prompt generated by the Auto-prompter. These configurations allow use to examine the impact of Auto-prompter and Planner on D-CIPHER, while also demonstrating the framework’s flexibility to build systems for different problems.

Table III shows the results for these two configurations. D-CIPHER without Auto-prompter with Claude 3.5 Sonnet gets a 3% improvement in challenges solved on NYU CTF Bench, but its performance drops with GPT 4o on NYU CTF Bench and Claude 3.5 Sonnet on Cybench, showing that the Auto-prompter improves performance in most cases. Without the Auto-prompter, average cost increases across LLMs and benchmarks, indicating that the Auto-prompter improves system efficiency without compromising performance in most cases. The contrasting result with Claude 3.5 Sonnet on NYU CTF Bench is due to the pwn category, where performance increases by more than 2×, while other categories get matching or lower results (see Sections VI-A and V-C).

D-CIPHER without Planner sees a 1% to 5% drop in performance on NYU CTF Bench across both LLMs. This highlights the benefit of the Planner-Executor system in solving CTF challenges. Despite the performance drop, the total cost of a Planner and multiple Executors is only 2× higher than a single Executor, showing that each individual agent is more efficient.

2) *Combining stronger and weaker LLMs*: D-CIPHER offers freedom to use different LLMs for each agent, and we experiment by combining stronger models for the Planner with

TABLE IV
DIFFERENT LLMs FOR PLANNER AND EXECUTOR.

Planner	Executor	% solved	\$ cost
Claude 3.5 Sonnet	Claude 3.5 Haiku	13.0	0.33
GPT 4o	GPT 4o mini	6.5	0.03
GPT 4 Turbo	GPT 4o mini	5.5	0.07
Gemini 1.5 Flash	Gemini 1.5 Flash 8B	3.0	0.001
LLaMa 3.1 405B	LLaMa 3.3 70B	0.0	0.00

weaker models for the Executor. The results are in Table IV showing consistent under-performance with weaker models. Claude 3.5 Sonnet with Haiku showed a 6.0% drop compared to Claude 3.5 Sonnet with Sonnet. Similarly, GPT-4o and GPT-4 Turbo, when paired with GPT-4o-mini, showed reductions of 4% and 1%, respectively. LLaMA 3.1 405B paired with LLaMA 3.3 70B failed to solve any challenges. Notably, Gemini maintained similar performance with the weaker. These results indicate that both the Planner and Executor tasks are complex to require stronger models.

3) *Impact of temperature*: D-CIPHER with GPT 4o is evaluated under a lower temperature setting of $T = 0.95$, with results in Table V. Decreasing the temperature show consistent drop across crypto, pwn, and rev with no improvements in forensics, web, or misc. Higher temperature offers creative and generative capabilities, helping problem-solving.

TABLE V
GPT 4o % solved FOR TEMPERATURES 0.95 AND 1.0.

	crypto	foren.	pwn	rev	web	misc	total
$T = 1.0$	5.8	13.3	7.7	13.7	10.5	16.7	10.5
$T = 0.95$	3.8	13.3	5.1	11.8	10.5	16.7	9.0

4) *Older LLM Versions*: As previously mentioned, we evaluate with latest versions of LLMs that show a natural improvement on CTF benchmarks, whereas EnIGMA evaluates with older versions. Table VI shows performance on NYU CTF Bench with the older Claude 3.5 Sonnet (claude-3-5-sonnet-20240620), same as EnIGMA. D-CIPHER outperforms EnIGMA but with almost 2× the cost. While this demonstrates the advantage of multi-agent systems, it also underlines the significance of the LLM’s capabilities.

TABLE VI
RESULTS WITH OLDER CLAUDE VERSION.

	% solved	\$ cost
EnIGMA	13.5	0.35
D-CIPHER	15.0	0.62

E. Exit Reason Analysis

Figure 6 shows the percentage breakdown of the challenge termination (exit) reasons of D-CIPHER on NYU CTF Bench. Exit reasons are of five types: “Solved” when the challenge is solved, “Giveup” when the Planner gives up, “Max cost” when the cost budget is exceeded, “Max rounds” when the Planner

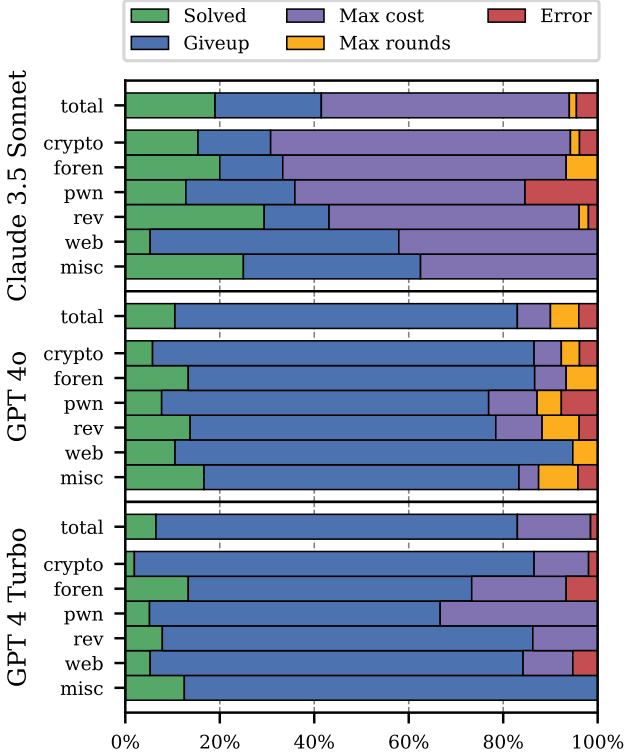


Fig. 6. % breakdown of exit reasons for D-CIPHER on NYU CTF Bench.

conversation rounds are exhausted, and “Error” when the run terminates with an error. For Claude 3.5 Sonnet, max cost is the most dominant exit reason, indicating less propensity to giveup but instead to continue with the challenge till the cost is exhausted. Comparatively, other LLMs have giveup as the most dominant reason. Max rounds are exhausted for only a few challenges. Distribution of exit reasons for GPT 4o and GPT 4 Turbo is similar across categories which shows the holistic capabilities of these models. Claude 3.5 Sonnet sees a high giveup and low success percentage on web challenges, highlighting a gap in capabilities. Examples of common failure reasons are present in Section VI-B.

F. Total Conversation Rounds Analysis

Figure 7 shows a histogram of the total conversation rounds of all agents in D-CIPHER. Successful challenges take lesser rounds than failed challenges, which may indicate that D-CIPHER only solves easier challenges requiring lesser rounds, but fails on longer challenges. However, it may also indicate that challenges are only solved when the correct path is found early enough, else the agents stray from the goal for many rounds before giving up. Claude 3.5 Sonnet runs for more rounds compared to GPT 4o and GPT 4 Turbo for both success and failure cases, re-iterating it’s propensity to keep going and not give up, which likely helps it solve challenges requiring many rounds. Looking at the Auto-prompter’s impact, we see that it helps solve more challenges faster, increasing efficiency.

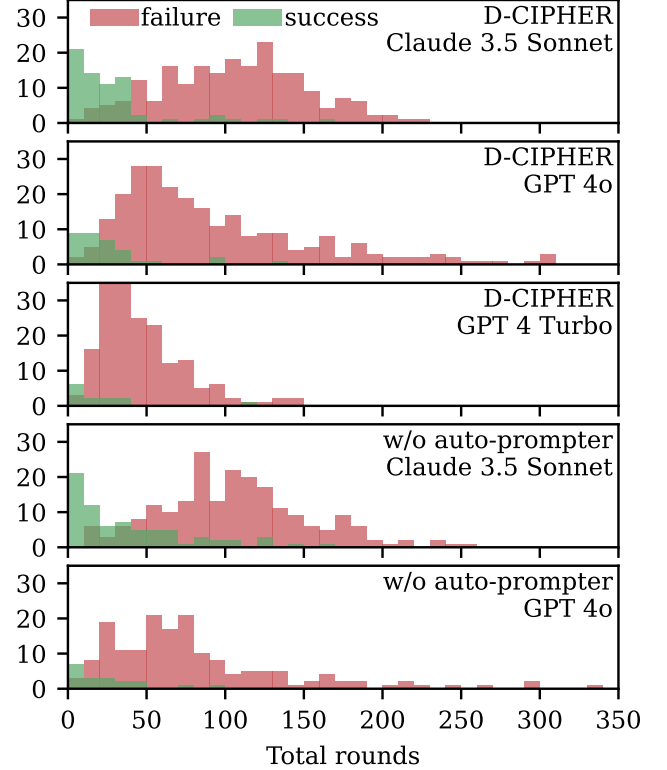


Fig. 7. Histogram of successful and failed challenges by total conversation rounds for D-CIPHER on NYU CTF Bench.

G. MITRE ATT&CK Capabilities

As described in Section IV-D, we labeled the 200 CTFs in NYU CTF Bench with MITRE ATT&CK techniques for elaborate analysis into D-CIPHER’s and related agents’ offensive capability. Table VII shows the breakdown of ATT&CK techniques that apply and how well each agent and each LLM performs on them. The “#CTFs” column shows the number of CTFs labeled with a technique along with a red heatmap to show higher counts. The agent and model columns show how many CTFs were solved by that agent for a particular technique, along with a blue heatmap to show higher count computed across all agents. If an agent solves a CTF mapped to multiple techniques, we consider that the solution has employed all techniques and we increment each count.

The results show that D-CIPHER without auto-prompter using Claude 3.5 Sonnet exhibits superior offensive capability as it solves 65% more techniques compared to other agents and configurations. Category-wise results show that D-CIPHER with Auto-prompter is weaker on pwn. The performance drops on multiple techniques spanning different categories, offering an insight into the Auto-prompter’s impact. Comparing D-CIPHER, NYU CTF Baseline, and EnIGMA on Claude 3.5 Sonnet, we see subtle but meaningful differences. D-CIPHER is better at T1110 (Brute Force) and T1600 (Weaken Encryption) as multi-agent collaboration aids in cryptographic CTFs, while EnIGMA outperforms on T1203 (Exploitation for Client Execution) and T1574 (Hijack Execution Flow) as interactive tools help for binary exploitation.

TABLE VII
ANALYSIS OF THE MITRE ATT&CK TECHNIQUES EMPLOYED BY EACH AGENT ON THE NYU CTF BENCH.

ID	Technique	#CTFs	D-CIPHER				NYUCTF Baseline			EnIGMA	
			Sonnet 3.5	GPT 4o	GPT4 Turbo	Sonnet w/o autoprompt	Sonnet 3.5	GPT 4o	GPT4 Turbo	Sonnet 3.5	GPT 4o
T1203	Exploitation for Client Execution	36	4	2	1	10	2	1	1	6	2
T1574	Hijack Execution Flow	24	2	1	0	5	0	0	0	3	1
T1190	Exploit Public-Facing Application	17	1	2	1	2	1	0	0	0	1
T1552	Unsecured Credentials	16	5	3	2	6	5	1	3	5	2
T1059	Command and Scripting Interpreter	15	1	1	1	3	1	1	1	1	1
T1110	Brute Force	11	3	0	0	3	3	1	2	1	2
T1600	Weaken Encryption	9	2	0	0	2	2	0	0	1	1
T1140	Deobfuscate/Decode Files or Information	9	1	0	0	2	1	0	0	1	1
T1055	Process Injection	7	1	0	0	1	0	0	0	1	0
T1212	Exploitation for Credential Access	6	0	0	0	0	0	0	0	0	0
T1027	Obfuscated Files or Information	6	1	0	0	2	1	0	0	2	1
T1083	File and Directory Discovery	5	2	2	1	2	1	0	0	1	2
T1071	Application Layer Protocol	4	0	0	0	0	0	0	0	0	0
T1539	Steal Web Session Cookie	3	0	0	0	0	0	0	0	0	0
T1001	Data Obfuscation	3	0	1	0	0	0	0	0	0	0
T1213	Data from Information Repositories	3	1	0	0	1	1	0	0	1	0
T1040	Network Sniffing	3	1	1	1	1	1	0	0	1	1
T1068	Exploitation for Privilege Escalation	2	0	0	0	0	0	0	0	0	0
T1497	Virtualization/Sandbox Evasion	2	0	0	0	1	0	0	0	0	0
T1005	Data from Local System	2	0	0	0	0	0	0	0	0	0
T1606	Forge Web Credentials	2	0	0	0	0	0	0	0	0	0
T1006	Direct Volume Access	2	1	0	1	1	1	0	0	1	1
T1505	Server Software Component	2	0	0	0	0	0	0	0	0	0
T1102	Web Service	1	0	0	0	0	0	0	0	0	0
T1556	Modify Authentication Process	1	0	0	0	0	0	0	0	0	0
T1078	Valid Accounts	1	0	0	0	0	0	0	0	0	0
T1614	System Location Discovery	1	0	0	0	0	0	0	0	0	0
T1082	System Information Discovery	1	0	0	0	0	0	0	0	0	0
T1649	Steal or Forge Authentication Certificates	1	0	0	0	0	0	0	0	0	0
T1565	Data Manipulation	1	0	0	0	0	0	0	0	0	0
T1033	System Owner/User Discovery	1	0	0	0	0	0	0	0	0	0
T1048	Exfiltration Over Alternative Protocol	1	0	0	0	0	0	0	0	0	0
T1555	Credentials from Password Stores	1	0	0	0	0	0	0	0	0	0
T1120	Peripheral Device Discovery	1	0	0	0	0	0	0	0	0	0
T1087	Account Discovery	1	0	0	0	0	0	0	0	0	0
T1106	Native API	1	0	0	0	0	0	0	0	0	0
T1593	Search Open Websites/Domains	1	0	0	0	0	0	0	0	0	0
T1542	Pre-OS Boot	1	0	0	0	0	0	0	0	0	0
T1486	Data Encrypted for Impact	1	0	0	0	0	0	0	0	0	0
T1003	OS Credential Dumping	1	1	1	0	1	1	0	1	1	0
T1553	Subvert Trust Controls	1	0	0	0	0	0	0	0	0	0
T1185	Browser Session Hijacking	1	0	0	0	0	0	0	0	0	0
T1036	Masquerading	1	0	0	0	0	0	0	0	0	0
T1133	External Remote Services	1	0	0	0	0	0	0	0	0	0
T1221	Template Injection	1	0	0	0	0	0	0	0	0	0
Total		211	27	14	8	43	21	4	8	26	16

D-CIPHER and EnIGMA solve similar number of techniques, even though D-CIPHER has 5.5% higher overall accuracy on NYU CTF Bench. This indicates that D-CIPHER is better at CTFs involving other skills apart from ATT&CK techniques, such as some reverse engineering and miscellaneous CTFs among the 83 CTFs not tagged with techniques. Similarly, NYU CTF Baseline and EnIGMA have similar overall accuracy, but NYU CTF Baseline solves less techniques, indicating weaker offensive capability. With GPT 4o, EnIGMA shows more uniform performance across techniques as compared to D-CIPHER and NYU CTF Baseline, which indicates that single agents with interactive tools may be more suitable for this model. D-CIPHER and the NYU CTF Baseline perform worse with GPT-4 Turbo, in line with the lower overall accuracy of this model.

D-CIPHER’s results with and without autoprompter compared to NYU CTF Baseline and EnIGMA show that multi-agent collaboration improves offensive security capability. This benchmarking of each agent’s offensive capability in terms of MITRE ATT&CK techniques has offered a nuanced perspective and an elaborate comparison metric. The composition of performance on ATT&CK techniques highlights the gaps and provides a guideline for future improvements.

VI. DISCUSSION

A. Auto-prompter Failures

As discussed in Section V-D1, D-CIPHER with Auto-prompter on Claude 3.5 Sonnet performs worse on pwn challenges of NYU CTF Bench compared to D-CIPHER without

Auto-prompter. We look at the five pwn challenges where D-CIPHER succeeds without Auto-prompter but fails with it.

slithery: A python jail escape challenge. The challenge server allows executing python code but maintains a reject list of commands. The solution bypasses the reject list to invoke python’s `os.system` for shell access. While the Auto-prompter understood the CTF’s purpose, a misleading base64 encoding threw the Auto-prompter off. It generated a prompt that focuses on the wrong variables, distracting the Planner.

unlimited_subway: buffer overflow. The solution involves leaking the stack canary byte-by-byte using arbitrary memory reads, exploiting a buffer overflow to overwrite the canary, and redirecting execution to the `print_flag` function. The Auto-prompter attempted to run commands such as `strings` to understand the binary, but continually encountered errors, ultimately failing to generate a useful prompt for the Planner.

got_milk: A global offset table attack. The solution exploits a format string vulnerability to overwrite the least significant byte of the global offset table address of the function `lose` with the corresponding byte of the function `win`, redirecting execution to the desired function. Auto-prompter could not extract any contextual information of the challenge and failed to generate a usable prompt, stalling the Planner.

bigboy: Another buffer overflow challenge. The solution involves exploiting a buffer overflow by overwriting a specific memory value with `0xCAF3BAEE` multiple times to pass the check and execute the `/bin/bash` command. Auto-prompter correctly analyzed the binary’s properties, behavior, and vulnerabilities and generated a prompt outlining the exploitation strategy, including payload construction and execution to solve the challenge. Despite this, the Planner failed.

baby_boi: Another buffer overflow challenge. The solution involves leveraging a buffer overflow to execute an ROP chain that reveals the `libc` base, locates one gadgets, and jumps to it to spawn a shell and retrieve the flag. The Auto-prompter generates a step-by-step prompt for exploiting the buffer overflow vulnerability, leveraging the `printf` overflow, and building a ROP chain to retrieve the flag. While the generated prompt covers all the necessary steps, it does not provide detailed information due to which the Planner fails.

From the five cases, we observe that while the Auto-prompter helps, it may be making D-CIPHER more susceptible to initial errors that the hard-coded prompt template may be robust to. These limitations lead to missed opportunities to fully exploit the challenge’s vulnerabilities or generate comprehensive and actionable prompts. Future work may combine Auto-prompter generated prompt with hard-coded guidelines.

B. Common Failure Examples

We inspected D-CIPHER’s conversation logs to document the common errors that led to failure.

Auto-prompter fails to generate prompt: Often, the Auto-prompter keeps running commands and exhausts maximum rounds without generating a prompt, even after being prompted one last time to call `GeneratePrompt`. In this case, we start the Planner with the hard-coded prompt template.

Agent produces no action: The agent’s response does not contain an action but only reasoning, usually if it is stuck

AUTO-PROMPTER GeneratePrompt

Please use `run_command` tool to explore the challenge, and generate the CTF challenge prompt to solve the challenge using the `generate_prompt` tool."

GeneratePrompt

"prompt": null

Use hard-coded prompt

PLANNER

You are a skilled cybersecurity professional and top CTF player. You are part of a CTF team in which your role is of the Planner. The other player in your team is also a skilled cybersecurity professional like you, and has the role of the..

Fig. 8. Auto-prompter generates empty prompt; fallback to hard-coded one.

and thinks that it needs user input despite being prompted to operate autonomously. This happens frequently with LLaMa 3.1 405B and Gemini 1.5 Flash which produce wrong syntax for function calls so the action is not parsed, as seen in Figure 9. In this case, we prompt the agent to retry.

PLANNER

```
<function=run_command>{"command\": \"file
thoroughlyStripped --preserve-dates -d -M -P -k -F -\"}\"/>  
</function>
```

 Observation

Please run a command to explore the challenge, or delegate the next task to the Executor using the `delegate` tool. If you think that you cannot proceed, give up on the challenge.

PLANNER

```
<function=run_command>{"command\": \"strings
thoroughlyStripped\"}\"/>  
</function>
```

 Observation

Please run a command to explore the challenge, or delegate the next task to the Executor using the `delegate` tool. If you think that you cannot proceed, give up on the challenge.

Fig. 9. Function call is not parsed correctly due to a formatting error by the LLM. The parsing error is returned and the LLM can try again.

Hallucinates CTF information: In some cases, agents try to connect to non-existent servers or read non-existent files as seen in Figure 10. Gemini 1.5 Flash also sometimes hallucinates functions that were not provided in the framework. Running these actions returns errors (e.g., “File not found”) that the agent must understand and rectify.

Confusion with interactive tools: This happens when an agent tries to run commands inside interactive tools like “gdb” but via `RunCommand` which only runs shell commands. A typical user would type these commands in an interactive shell in this manner, but the agent does not have an interactive interface. Advanced interactive tools and demonstrations for awareness of the agent’s interface may help fix such errors.